

EDUGRAPH AI & SCHOOL BOX

8-Week Implementation Plan

Pilot deployment across 2-3 Ethiopian schools

Curriculum intelligence • Exam-CLO alignment • Calibrated
learning-gap analysis

Delivery horizon: 40 working days over 8 weeks

Deployment model: Central cloud + offline School
Box edge nodes

Executive Summary

This document presents a detailed day-by-day, week-by-week, and month-by-month implementation plan to deploy **EduGraph AI** and the on-premise **School Box (Edge Node)** into 2-3 pilot schools in Ethiopia over an 8-week period.

Implementation Architecture

1	Curriculum Ingestion & Graph Decomposition	Convert curriculum files into structured Subject → Unit → Topic → CLO knowledge graphs.
2	Exam-CLO Matching	Use semantic alignment to map exam questions to curriculum learning outcomes with teacher review.
3	Calibrated Gap Analysis Engine	Trace prerequisite gaps, discount uncertain signals, and generate bilingual study guidance.

Pillar & Model Architecture Matrix

Pillar 1: Curriculum Ingestion	Document Layout & Parser	PyMuPDF + DOCX extractor	Read-only pre-loaded graph mirror
Pillar 2: Exam-CLO Matching	Semantic Vector Alignment	BAAI/bge-m3 (ONNX INT8 Quantized)	CPU-optimized local inference
Pillar 3: Gap Analysis	Graph Traversal & Insights	Neo4j Cypher + Ollama (qwen2.5 / llama3.2)	Instant local Cypher + local LLM

Month 1: Core Pillar Hardening & Edge Engine

Week 1: Pillar 1 - Curriculum Ingestion & Storage Adapter

Day 1 (Mon)	Audit the local stack (api, ai-service, postgres, neo4j, redis, ollama). Implement server-side magic-byte MIME validation (sniffCurriculumMime) in Go.
Day 2 (Tue)	Configure the PostgresStorage adapter (storage.local_files BYTEA) for zero-S3 reliance on School Boxes. Refine multi-column extraction in PyMuPDF/DOCX extractors.
Day 3 (Wed)	Connect parsing tasks to Redis queue:curriculum:parse. Ensure structure JSONB generates Subject → Unit → Topic → CLO hierarchies.
Day 4 (Thu)	Validate JobReviewPage.tsx so curriculum officers can edit tree structures. Implement Bearer-token blob fetching for secure document previews.
Day 5 (Fri)	Execute single-transaction SQL promotion (ApproveAndPromote) and mirror nodes into Neo4j: Subject -HAS_UNIT→ Unit -HAS_TOPIC→ Topic.

Week 2: Pillar 2 - Exam-CLO Matching & Quantized CPU Embeddings

Day 6 (Mon)	Configure the queue:exam:parse worker in the Python AI Service to extract question text, options, total marks, and estimated times.
Day 7 (Tue)	Configure the queue:exam:answerkey worker to align answer keys with CLO IDs in assessment.questions.
Day 8 (Wed)	Export the BAAI/bge-m3 embedding model to ONNX INT8 quantized format for fast CPU execution on School Box hardware.
Day 9 (Thu)	Build semantic cosine-similarity matching. Thresholds: ≥ 0.85 auto-accept; 0.65-0.84 teacher verification; < 0.65 reject.
Day 10 (Fri)	Verify ExamReviewPage.tsx for teacher review and confirmation of auto-suggested CLO pairings before test publication.

Week 3: Pillar 3 - Calibrated Gap Analysis Engine & Local LLM

Day 11 (Mon)	Update exam_submit.go to log student choices and timeSpentSecs. Track exposure_count per student-topic.
Day 12 (Tue)	Build a 3-pass graph-analysis worker (queue:gap:analyze): triage missed items, walk upstream Neo4j prerequisites 1-3 hops, and require exposure_count ≥ 1 for true root causes.
Day 13 (Wed)	Classify untested upstream topics as "Unverified Prerequisites." Apply a 25% base MCQ guess discount to single-attempt tests.
Day 14 (Thu)	Wire a local Ollama runner (qwen2.5:3b-instruct / llama3.2:3b) into ai-service for offline bilingual English/Amharic insight generation.
Day 15 (Fri)	Finalize study_plan_worker.py (queue:studyplan:generate): run Kahn's topological sort over prerequisite DAGs to order daily study targets.

Week 4: Multi-Tenant Security & Outbox Sync Pattern

Day 16 (Mon)	Audit Go SQL handlers and Cypher queries to enforce strict <code>school_id</code> isolation across all database calls.
Day 17 (Tue)	Configure JWT RS256 offline signing-key generation (<code>APP_ENV=production_edge</code>). Enable 14-day offline refresh-token rotation.
Day 18 (Wed)	Implement <code>public.sync_logs</code> and <code>outbox_events</code> tables in the Go backend for store-and-forward telemetry sync.
Day 19 (Thu)	Connect <code>GET /api/v1/teachers/me/class-heatmap</code> . Trigger cross-grade warnings in <code>HeatmapGrid.tsx</code> when more than 40% of a class shares an unmastered prerequisite gap.
Day 20 (Fri)	Enable 1-hour Redis caching (<code>school_quality:{schoolId}</code>) and test the nightly background metric-computation worker.

Month 2: School Box Packaging, Simulation & Field Rollout

Week 5: School Box Containerization & Hardware Staging

Day 21 (Mon)	Create <code>docker-compose.schoolbox.yml</code> with explicit CPU/RAM caps for Postgres, Neo4j, Redis, and the Python AI Service. Set <code>restart: always</code> .
Day 22 (Tue)	Disconnect internet access and verify complete offline functionality: login, exam taking, gap traversal, study plans, and Graph-RAG tutoring.
Day 23 (Wed)	Configure daily shell cron jobs for automated Postgres (<code>pg_dump</code>) and Neo4j database dumps to an attached external USB drive.
Day 24 (Thu)	Install Ubuntu Server LTS on three physical mini-PC/server units ("School Boxes"). Pre-install Docker, systemd services, and local LAN networking.
Day 25 (Fri)	Run <code>seed_curriculum.py</code> to pre-load Ethiopian Grade 9-12 Mathematics, Physics, and Chemistry curricula into each local School Box database.

Week 6: LAN Stress Testing, Sync Dry-Run & Polish

Day 26 (Mon)	Run k6 load tests simulating 150 concurrent student exam submissions over a local school Wi-Fi testbed.
Day 27 (Tue)	Simulate three days of offline exam activity. Attach a 4G dongle and trigger background sync (<code>internal/sync</code>) to push outbox events to the Central Cloud.
Day 28 (Wed)	Confirm UUID primary keys prevent record collisions when syncing data from multiple pilot schools into the Central Cloud.
Day 29 (Thu)	Build a local admin health page (<code>/admin/school-box</code>) showing disk usage, queue depth, and sync-status logs.
Day 30 (Fri)	Polish responsive UI behavior on low-cost student tablets and audit Amharic translation strings across dashboards.

Week 7: On-Site Installation, School Setup & Staff Training

Day 31 (Mon)	Transport and install School Boxes in pilot School 1 and School 2 computer labs. Configure the LAN to resolve <code>http://schoolbox.local:8080</code> .
Day 32 (Tue)	Install the School Box in pilot School 3. Perform LAN smoke tests across client PCs and tablets.
Day 33 (Wed)	Conduct hands-on teacher workshops covering paper-exam uploads, review of auto-matched CLOs, and digital test configuration.
Day 34 (Thu)	Train teachers to interpret class-wide gap heatmaps. Train school administrators on offline health monitoring and user management.
Day 35 (Fri)	Run bulk CSV imports to register student accounts and link them to schools, grades, and sections.

Week 8: Field Go-Live & Operational Support

Day 36 (Mon)	OFFICIAL PILOT GO-LIVE. Launch the system across 2-3 pilot schools. Students log into local School Box portals in computer labs.
Day 37 (Tue)	Observe the first live digital-exam session. Monitor Redis queue : gap : ana lyze to verify instant student feedback.
Day 38 (Wed)	Audit generated students . gap _records and diagnostic insights for accuracy with participating teachers.
Day 39 (Thu)	Connect pilot School Boxes to external network links to trigger outbox-event synchronization with the Central Cloud.
Day 40 (Fri)	Hold review meetings with school principals and teachers to collect feedback and evaluate the Phase 1 pilot deployment.

Pilot Readiness Gates

Platform readiness	All core services run under constrained School Box resources with restart policies and offline login.
Academic readiness	Curriculum graph is approved; exam-CLO mappings are teacher-verifiable; study plans follow prerequisite order.
Security readiness	School-level isolation is enforced in SQL and Cypher; RS256 keys and offline token rotation are configured.
Operational readiness	Backups, admin health monitoring, queue visibility, and outbox synchronization are tested.
Field readiness	Hardware is installed, users are imported, staff are trained, and a live exam is observed successfully.

Target outcome: a secure, offline-capable, academically calibrated pilot that can operate locally and synchronize safely whenever connectivity becomes available.